

RAG Powered AI Assistant

Python

Streamlit

FAISS

Ollama

Mistral

Complete Development Journey, Architecture,
Setup Guide, Code Explanation & Learning Handbook

Created by

Devansh Panwar

Table of Contents

1. Introduction
2. What is RAG?
3. Why This Project Was Built
4. Technologies Used
5. Complete Project Architecture
6. Phase 1 — Core RAG Pipeline
7. Phase 2 — Persistent Vector Database
8. Phase 3 — Frontend Development
9. Phase 4 — Production Features
10. Challenges Faced
11. Performance Optimization
12. Learning Outcomes
13. Industry Use Cases
14. Future Improvements
15. Complete Setup Guide
16. Conclusion

1. Introduction

Artificial Intelligence is rapidly changing the way businesses work with data and documents. Traditional systems require manual searching through PDFs, contracts, reports, notes, and files, which consumes huge amounts of time. This project was built to solve that problem using Retrieval-Augmented Generation (RAG).

The system is capable of:

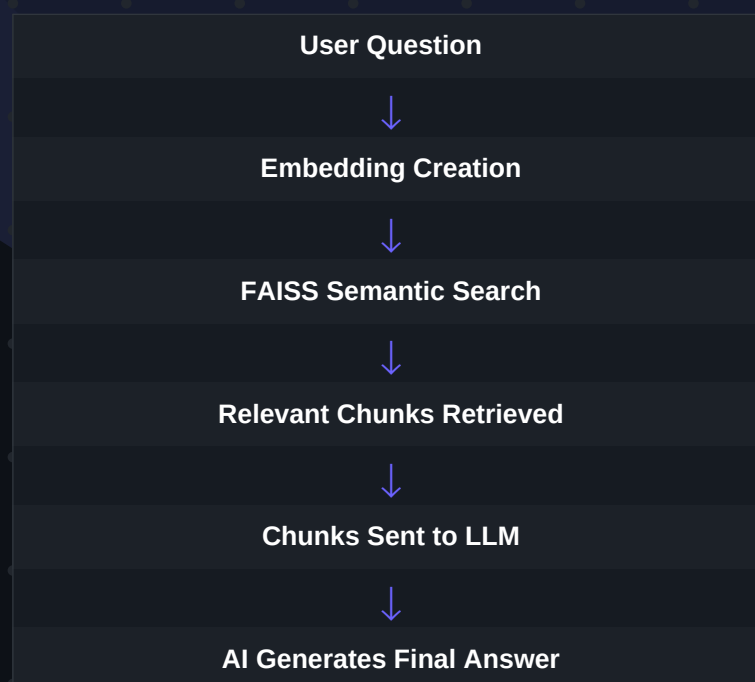
- ✓ Reading PDFs
- ✓ Understanding document meaning
- ✓ Searching semantically
- ✓ Retrieving relevant information
- ✓ Generating intelligent answers
- ✓ Working completely locally without paid APIs

Unlike traditional chatbots, this system answers directly from uploaded documents. It demonstrates modern AI engineering concepts including vector databases, embeddings, semantic search, local LLM deployment, prompt engineering, AI pipelines, and frontend AI integration.

2. What is RAG?

RAG stands for **Retrieval-Augmented Generation**. Traditional Large Language Models generate responses only from data learned during training — they cannot access private documents, they hallucinate, and they cannot search custom files. RAG solves this by adding a Retrieval System, Document Search, and Context Injection layer.

RAG Workflow





3. Why This Project Was Built

Professionals deal with thousands of pages of documents daily. Manual search is slow, traditional keyword search is weak, paid APIs are expensive, and privacy risks exist when sending documents to cloud services.

Lawyers	Doctors	CA Firms	HR Depts
Contract analysis	Medical report reading	Tax document retrieval	Policy searching

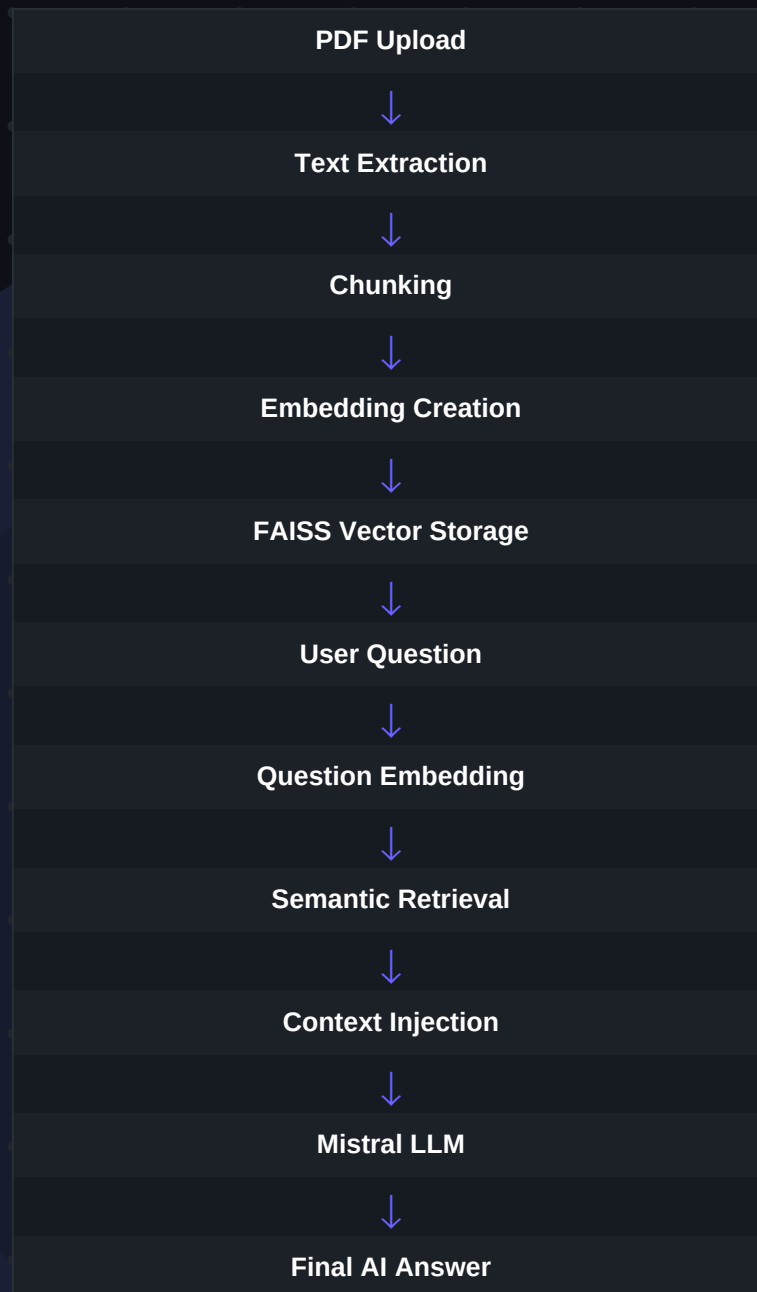
This project solves:

- ✓ Faster search
- ✓ Semantic understanding
- ✓ AI-powered retrieval
- ✓ Local execution
- ✓ Zero API cost
- ✓ Better productivity

4. Technologies Used

Python	Main language for AI pipeline, backend logic, and PDF processing	Huge AI ecosystem, easy integration, rich libraries
Streamlit	Frontend — upload PDFs, ask questions, display AI responses	Fast development, Python-based, easy deployment
FAISS	Store vectors and perform semantic search (Facebook AI Similarity Search)	Extremely fast, lightweight, runs locally
Sentence Transformers	Create embeddings using all-MiniLM-L6-v2 model	Small, fast, accurate for semantic tasks
Ollama	Run large language models locally on your machine	Free, offline, zero API costs
Mistral	Generate final AI answers from retrieved context	Better reasoning, better responses, lightweight

5. Complete Project Architecture



6. Phase 1 — Core RAG Pipeline

Objective: Build a basic Retrieval-Augmented Generation system.

PDF Loading

Extract text from uploaded PDFs

`PdfReader()`

Chunking

Break large text into small chunks — LLMs cannot process huge documents at once

`RecursiveCharacterTextSplitter`

Embeddings

Convert text chunks into numerical vectors

```
SentenceTransformer
```

FAISS Storage

Store vectors for fast semantic search and retrieval

```
faiss.IndexFlatL2
```

Problem Encountered

ModuleNotFoundError — wrong virtual environment was active.

Fix: `venv\Scripts\activate`

7. Phase 2 — Persistent Vector Database

Objective: Store vectors permanently so PDFs do not need re-processing on every run.

File	Purpose
<code>faiss.index</code>	Stores the FAISS vector index
<code>chunks.pkl</code>	Stores original text chunks for retrieval
<code>metadata.pkl</code>	Stores source file and page number for citations

Problem: metadata.pkl missing error on startup. Cause: old vector database from previous build. Fix: delete vectorstore/ folder and rebuild with `python build_index.py`

8. Phase 3 — Frontend Development

Objective: Build an interactive web UI for non-technical users.

Features Added:

- ✓ Upload PDFs via sidebar
- ✓ Ask questions in a chat interface
- ✓ Display AI responses with sources
- ✓ Conversational memory across turns
- ✓ Source citations with page numbers

Run command:

```
streamlit run streamlit_app.py
```


9. Phase 4 — Production Features

- ✓ Multi-PDF support — process multiple documents in one session
- ✓ Metadata tracking — know which file and page an answer came from
- ✓ Conversational memory — context persists across questions
- ✓ Better prompts — structured prompt templates for more accurate answers
- ✓ Faster retrieval — optimized FAISS search parameters
- ✓ Source citations — show exact document references in UI

10. Challenges Faced

Error	Cause	Fix
LangChain Import Errors	Version conflicts between langchain packages	<code>pip install langchain-text-splitters</code>
Ollama Not Recognized	Ollama binary not installed or not on PATH	Install Ollama, restart terminal
Slow Responses	CPU-only inference with large chunk sizes	Reduce chunk count, smaller prompts
Streamlit Blank Screen	Backend crash — silent failure in terminal	Check terminal logs carefully

11. Performance Optimization

- ✓ Reduced chunk count per query from 5 to 3
- ✓ Switched to smaller embedding model (all-MiniLM-L6-v2)
- ✓ Improved prompt engineering — shorter, more precise context windows
- ✓ Reused loaded models across sessions — no re-initialization overhead
- ✓ Cached FAISS vectors to disk — no recomputation on restart

12. Learning Outcomes

AI Concepts	Engineering	Deployment
RAG systems Semantic search Embeddings Vector databases	Debugging Dependency management Frontend integration Production pipelines	Streamlit cloud local serving integration Ollama LLM Model optimization

13. Industry Use Cases

Legal Firms

Contract analysis, clause extraction, precedent search across case files

Healthcare

Medical note analysis, patient history retrieval, drug interaction lookup

CA & Finance

Tax document retrieval, GST invoice analysis, audit report search

Enterprise

Internal knowledge assistants, HR policy chatbots, onboarding systems

14. Future Improvements

- ✓ FastAPI backend for scalable REST API serving
- ✓ Public cloud deployment with authentication
- ✓ Cloud GPU inference for faster responses
- ✓ User authentication and multi-tenant support
- ✓ OCR support for scanned and image-based PDFs
- ✓ Voice assistant interface
- ✓ Docker containerization for easy deployment
- ✓ Cloud vector databases (Pinecone, Weaviate)

15. Complete Setup Guide

Step 1 — Install Python

Download from <https://python.org> and install with "Add to PATH" checked

Step 2 — Create Virtual Environment

```
python -m venv venv
```

Step 3 — Activate Environment

```
venv\Scripts\activate (Windows) source venv/bin/activate (Mac/Linux)
```

Step 4 — Install Libraries

```
pip install streamlit faiss-cpu sentence-transformers pypdf ollama
```

Step 5 — Install Ollama

Download from <https://ollama.com> and install

Step 6 — Pull Mistral Model

```
ollama pull mistral
```

Step 7 — Build Vector Database

```
python build_index.py
```

Step 8 — Run the Application

```
streamlit run streamlit_app.py
```

16. Conclusion

This project began as a simple attempt to understand Retrieval-Augmented Generation. Over time, it evolved into a production-style AI system capable of reading documents, understanding semantic meaning, retrieving relevant information, and generating intelligent responses — all running locally with zero API cost.

This project demonstrates:

- ✓ AI engineering with real-world RAG architecture
- ✓ Vector databases and semantic search systems
- ✓ Local LLM deployment with Ollama and Mistral
- ✓ Frontend AI integration with Streamlit
- ✓ Production debugging and performance optimization
- ✓ End-to-end AI workflow from ingestion to answer generation

This is a strong foundation for future enterprise AI systems and demonstrates the skills most sought after in AI engineering roles in 2026.

Built by Devansh Panwar

RAG Powered AI Assistant — Complete Documentation